

BAUTISTA JENELLA KISSES B.

BSIT 3-1

QUALITY ASSURANCE: ACTIVITY 2

1. How do socio-technical factors contribute to software failures, and what strategies can be implemented to address them?

-Software failures are often caused not only by technical flaws but also by socio-technical factors, such as poor communication, lack of user involvement, and organizational culture. Important problems are those involving insufficient communication, exclusion of users and corporate culture. Insufficient communication between developers, managers, and stakeholders results in imprecise requirements and eventual flawed software. The best example for such would be Healthcare.gov, where such issues emerged due to failure in team cooperation. It is better for organizations to try out Agile methods to solve these problems. With this type of method, teams collaborate much better and get frequent feedback.

One more vital aspect is user involvement because applications without considering the end-user demands face quite usability problems that often point towards non-use or inefficient uses. One of the really important examples included the software problem with the Airbus A380, which resulted in delays due to inter-system version differences. Usability testing should occur at all stages of software development, where developers adopt user-centered design to mitigate such failure scenarios. It can be an example that emphasizes avoiding last-minute deployments and costly mistakes by creating a culture of quality and proper training such as the case of Knight Capital for trading software at its failure.

Software success is also influenced by security processes and ethics. One lesson learned from the Equifax data breach is that neglecting cybersecurity precautions can have serious and painful repercussions on companies involved. Organizations need to consider the implementation of proactive security measures like periodic audits and ethical coding standards. In brief, by integrating five socio-technical criteria that improve communication, give priority to user needs, shape a culture that values quality, and deliver ethical accountability, businesses can accomplish risk mitigation and enhancement of their software reliability.

2. How do socio-technical factors contribute to software failures, and what strategies can be implemented to address them?

-Technical debt represents concessions in software development that emphasize short-term gains over long-term worth accruing interest whenever developers take shortcuts, be it through writing inefficient code, skipping documentation, or delaying refactoring in order to meet deadlines or save costs. These trade-offs speed things up, yet oftentimes they carry with them hidden long-term consequences that make future maintenance, scalability, and performance hard. Unattended, technical debt could lead to a breakdown of software in the long run, rendering the systems un-upgradeable, potential security threats, and maintenance-intensive defects.

Rapid development cycled constitute a major source of technological debt accumulation. Organizations usually aim for faster releases without adequate time for testing and code optimization-a trend that slowly evolves into nicely structured codebases impossible to change, leading to a rise in defect rates and stability problems. Furthermore, additional debt arises from aging technologies and/or frameworks since keeping them compliant with modern solution standards becomes a nightmare in itself. This was quite evident during the Equifax data fiasco in 2017, where the inability to apply critical security patches laid open customers' sensitive data, showing how unattended technical debt can have far-reaching consequences.

Business strategies are to manage the technical debt so it does not cumulate into a long-term failure: continuous refactoring, automated testing, and technical debt tracking. Another major strategy involves applying DevOps principles for the close collaboration of the developers and operations teams, thus maintaining the quality of their code and minimizing accumulated debt. Also, it is recommended to cultivate an organizational culture that rewards long-term value as opposed to short-term deployments. Being proactive in managing technical debt with measures such as regular code reviews, refactoring, and updates to legacy systems will ensure stability, security, and maintainability long-term, thus reducing the odds of future failures from the onset.

3. How do systemic failures in software development lifecycles contribute to large-scale software project failures

-Systemic failures in the software development lifecycle (SDLC) might occur when various infinitesimal flaws in processes, methodologies, or management decisions alter them into accumulating inefficiencies over time. Systemic failures percolate through multiple crossroads: (1). Poor Planning, (2). Inadequate Requirement Gathering, (3) Communication with Stakeholders, and (4) Insufficient Testing, these weaknesses can

convert themselves into major contributors to software project failure and, thus, to budget overrun coupled with schedule slippage and unusable or, at the least, unstable software products. An excellent case study that depicts major failures of the FBI's virtual case-file system through mismanaged requirements, poor oversight on development, and unrealistic expectations was the eventual canceling of the project, with millions poured down into it.

One of the greatest reasons for which systemic failures escalate is lack of proper risk management and flexibility. Most organizations adhere to rigid SDLC models, irrespective of evolving user requirements and changing technologies, hence resulting in obsolete or deficient software. Consequently, miscommunications arise during these interactions, rendering development teams, stakeholders, and end-users uncertain about system functionality, allowing major defects to elude detection until late in the development life cycle. An infamous example of an SDLC process, whereby testing and documentation inadequacies led to threats to safety, is the Boeing 737 MAX software failure, which emphasizes the consequences of systemic flaws.

Another way to avert systemic failures is through adopting adaptive methodologies like Agile and DevOps that build continuous feedback, iterative development, and team collaboration. Investment into comprehensive requirement analysis, rigorous testing, and automated quality assurance processes ensures the early identification of defects, thereby reducing long-term risk. In addition, a culture of accountability in which teams are transparent and proactively assess risk will further minimize systemic inefficiencies on large-scale projects. By targeting the above foundational issues, software teams are in a better place to increase project success and mitigate catastrophic failures.